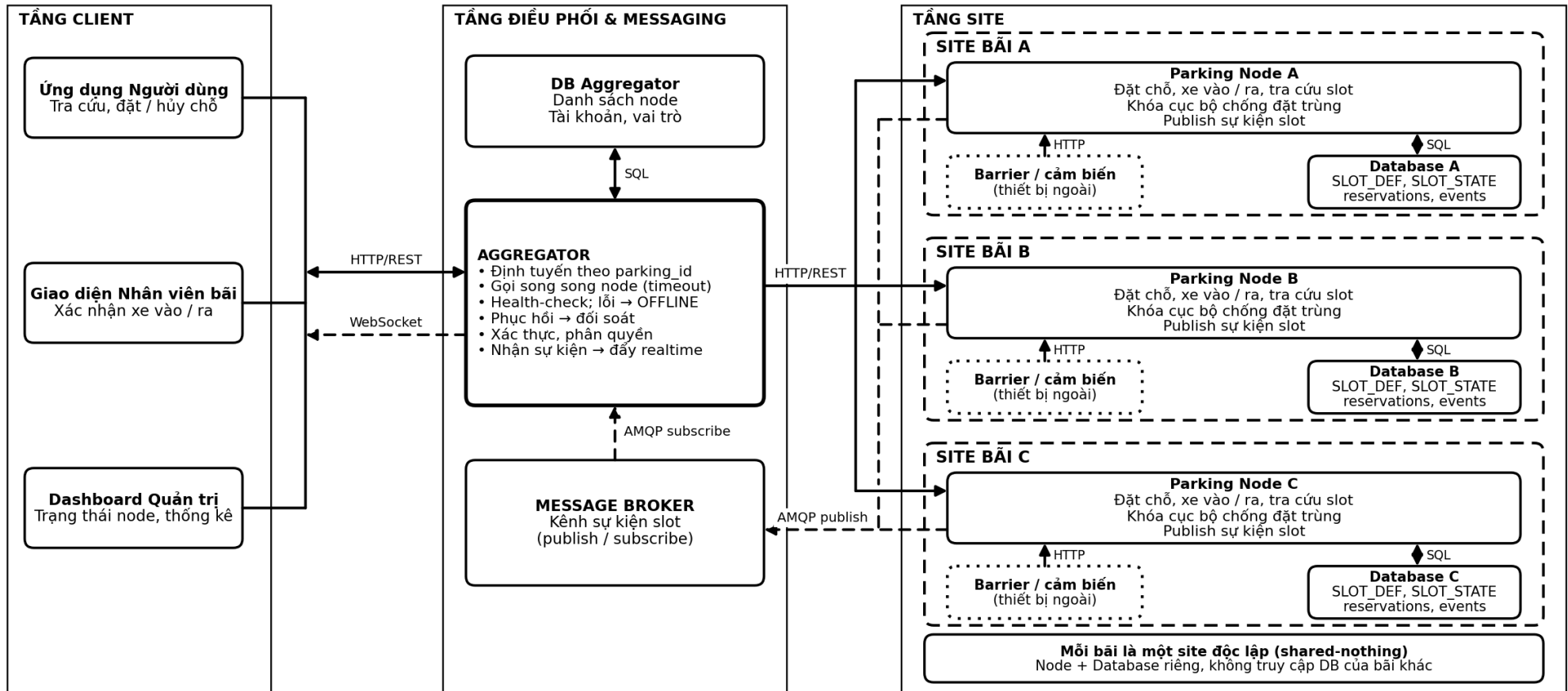


MÔ HÌNH HLD CỦA HỆ THỐNG PHÂN TÁN

Đề tài: Hệ thống bãi đỗ xe thông minh liên kết nhiều bãi • Môn: Hệ thống phân tán • **Nhóm 16** - PTIT

MÔ HÌNH HLD - HỆ THỐNG BÃI ĐỖ XE THÔNG MINH LIÊN KẾT NHIỀU BÃI

Nét liền: gọi đồng bộ • Nét đứt: sự kiện bất đồng bộ • Khung nét đứt lớn: ranh giới triển khai • Khung nét chấm: thiết bị ngoài



Hình 1. Mô hình HLD: ba tầng Client, Điều phối & messaging, Site; mỗi bãi là một site độc lập (khung nét đứt lớn là ranh giới triển khai).

Câu 1. Vì sao cần phân tán?

Trả lời: vì dữ liệu và nghiệp vụ của bài toán vốn phát sinh tại nhiều bãi độc lập. Gom tất cả về một máy chủ tập trung sẽ tạo ra điểm lỗi duy nhất, điểm nghẽn về tải, và làm mỗi bãi phụ thuộc vào đường mạng lên trung tâm cho cả những thao tác cần phản hồi ngay tại chỗ.

Bốn lý do cụ thể:

- **Dữ liệu phát sinh tại từng bãi.** Mỗi bãi có Barrier / cảm biến, nhân viên và tập slot riêng. Xe vào/ra cần được xác nhận và cập nhật trạng thái slot tức thì, nên xử lý và lưu trữ đặt ngay tại bãi (Node + Database riêng) thay vì chờ một máy chủ ở xa.
- **Chịu lỗi và cô lập lỗi.** Nếu tập trung, máy chủ hoặc CSDL chung hỏng thì mọi bãi ngừng hoạt động. Khi phân tán, bãi B mất kết nối thì bãi A và C vẫn cho xe vào/ra và nhận đặt chỗ; tra cứu vẫn trả kết quả của A, C và đánh dấu B: OFFLINE.
- **Mở rộng theo số bãi.** Thêm một bãi mới chỉ cần thêm một site và đăng ký vào danh sách node, không phải di chuyển hay sửa dữ liệu của các bãi cũ. Tải ghi (đặt chỗ, vào/ra) được chia theo bãi nên không tranh chấp trên một CSDL chung.
- **Tự chủ nhưng vẫn thống nhất với người dùng.** Mỗi bãi tự vận hành (khóa slot bảo trì, xác nhận xe) mà người dùng chỉ thấy một hệ thống duy nhất, vì Aggregator che giấu vị trí dữ liệu (tính trong suốt).

Tiêu chí	Nếu tập trung một máy chủ	Khi phân tán theo bãi
Khi có sự cố	Toàn hệ thống dừng	Chỉ bãi lỗi bị ảnh hưởng; bãi khác vẫn chạy
Tải ghi	Mọi bãi dồn vào một CSDL	Mỗi bãi ghi vào CSDL của mình
Thêm bãi mới	Sửa hệ thống chung, rủi ro cho các bãi cũ	Thêm site và đăng ký node, bãi cũ không đổi
Xe vào/ra	Phụ thuộc đường mạng lên trung tâm	Xử lý tại chỗ, không phụ thuộc bãi khác

Câu 2. Phân tán ở đâu?

Trả lời: hệ thống được phân tán ở bốn lớp: **dữ liệu, xử lý, triển khai** và **luồng thông điệp**. Mỗi bãi là một đơn vị độc lập gồm Node và Database riêng (shared-nothing); phần điều phối ở giữa chỉ định tuyến và tổng hợp.

Lớp	Thành phần trên sơ đồ	Phân tán cụ thể
Dữ liệu	Database A, B, C	Phân mảnh ngang theo parking_id: mỗi bãi một CSDL, mỗi bản ghi thuộc đúng một bãi. Phân mảnh dọc trong từng node: SLOT_DEF (định nghĩa slot, ít thay đổi) tách khỏi SLOT_STATE (trạng thái, thay đổi liên tục). reservations và events phân mảnh phải sinh theo slot nên một lượt đặt chỗ nằm trọn trong một node.
Xử lý	Parking Node A, B, C; Aggregator	Xử lý chia hai cấp. Cấp bãi: Node tự xử lý đặt chỗ, xe vào/ra, tra cứu slot và khóa cục bộ chống đặt trùng. Cấp điều phối: Aggregator định tuyến, gọi song song các node, tổng hợp kết quả, health-check, xác thực và phân quyền.
Dữ liệu điều phối	DB Aggregator	Danh sách node (địa chỉ, trạng thái) và tài khoản, vai trò được lưu riêng, không lẫn với dữ liệu của bãi.
Triển khai	Khung nét đứt quanh từng site	Mỗi site là một đơn vị triển khai độc lập (tiến trình / container, cổng và CSDL riêng), giao tiếp với nhau qua mạng bằng HTTP/REST, WebSocket, AMQP và SQL.
Luồng thông điệp	Message Broker	Slot đổi ở node nào thì node đó publish sự kiện; Aggregator subscribe rồi đẩy realtime xuống client. Node không cần biết ai đang theo dõi.
Thiết bị ngoài	Barrier / cảm biến	Đặt tại từng bãi, chỉ gọi vào Node của chính bãi đó qua HTTP.

Phần không phân tán (tập trung): Aggregator, DB Aggregator và Message Broker chỉ có một bản, nhưng không giữ dữ liệu slot nên việc tập trung này không ảnh hưởng tới tính tự chủ của các bãi.

Câu 3. Giải pháp phân tán là gì?

Trả lời: dùng mô hình **điều phối viên (Aggregator) + các site tự chủ**. Dữ liệu được phân mảnh theo bãi; yêu cầu một bãi được định tuyến thẳng tới node đó, yêu cầu toàn hệ thống được gọi song song có timeout; nhất quán mạnh trong từng site, cuối cùng ở toàn cục; lỗi được cô lập bằng health-check và trả kết quả một phần.

Vấn đề	Giải pháp
Tổ chức hệ thống	Aggregator là điểm vào duy nhất; client không gọi trực tiếp Node hay DB. Mỗi site độc lập, không truy cập DB của bãi khác.
Định vị dữ liệu	Phân mảnh theo parking_id. Aggregator tra danh sách node ở DB Aggregator rồi chuyển yêu cầu tới đúng node.
Yêu cầu một bãi (đặt chỗ, vào/ra)	Định tuyến thẳng tới node đích qua HTTP/REST; không đi qua các bãi khác.
Yêu cầu toàn hệ thống (tra cứu chỗ trống)	Scatter-gather: gọi song song các node có timeout, tổng hợp kết quả. Node lỗi hoặc chậm thì bỏ qua và trả kết quả một phần, kèm trạng thái của bãi đó.
Giao tiếp	Đồng bộ (HTTP/REST) cho tra cứu và đặt chỗ; bất đồng bộ (AMQP publish/subscribe) cho sự kiện trạng thái slot; WebSocket đẩy realtime xuống client; SQL giữa Node với DB của bãi và giữa Aggregator với DB Aggregator.
Nhất quán và đồng thời	Trong từng site: khóa cục bộ chống hai người cùng đặt một slot. Một lượt đặt chỗ chỉ nằm trong một node nên không cần giao dịch phân tán (2PC). Toàn cục: nhất quán cuối cùng vì trạng thái tổng hợp cập nhật qua sự kiện. Theo CAP: đặt chỗ ưu tiên nhất quán (từ chối khi node OFFLINE), tra cứu ưu tiên sẵn sàng (trả kết quả một phần).
Chịu lỗi	Aggregator health-check từng node; node lỗi bị đánh dấu OFFLINE, ghi vào node đó báo lỗi ngay (không xếp hàng ngầm). Node phục hồi thì Aggregator đổi soát lại trạng thái. Sự kiện được lưu trong bảng events của DB bãi rồi mới publish để không mất khi broker tạm lỗi.
Trong suốt	Người dùng chỉ thấy một hệ thống: không cần biết bãi nằm ở node nào, chạy trên máy nào; kết quả một phần được gắn trạng thái của bãi lỗi.
Bảo mật	Xác thực và phân quyền (USER, STAFF, ADMIN) ở Aggregator; lời gọi từ Aggregator xuống Node kèm khóa nội bộ.

Ví dụ minh họa

- **Đặt chỗ ở bãi B:** client gửi yêu cầu tới Aggregator, Aggregator xác thực và tra node B ở DB Aggregator, gọi node B; node B khóa cục bộ slot, ghi reservation vào DB B, phát sự kiện SLOT_UPDATED; Aggregator nhận sự kiện và đẩy realtime tới các client đang xem.
- **Tra cứu khi bãi C mất kết nối:** Aggregator gọi song song A, B, C; C không phản hồi trong timeout nên bị bỏ qua; client nhận kết quả của A, B cùng trạng thái C: OFFLINE. Khi C chạy lại, health-check phát hiện và Aggregator đổi soát trạng thái của C.

Hạn chế và hướng mở rộng. Aggregator, Message Broker và DB Aggregator là các điểm tập trung; hệ thống chưa dùng Load Balancer vì tải được chia theo dữ liệu (mỗi bãi một node) và Aggregator chỉ định tuyến, tổng hợp. Hướng mở rộng: nhiều bản Aggregator sau Load Balancer, cụm broker và bản sao DB điều phối.